Cross-Query Optimization for Worst-Case Optimal Joins in Apache Calcite

Thomas Banghart
tbanghar@ucsc.edu

Abstract

Cyclic join queries pose a challenge for binary join engines because no join ordering avoids intermediate results larger than the final output. Worst-case optimal join (WCOJ) algorithms solve this for single queries, but batches of similar cyclic queries still redundantly build identical data structures and traverse the same search space. We present the first system combining WCOJ with multi-query optimization, implemented as extensions to Apache Calcite. We introduce a `Combine` operator and `MULTI()` SQL syntax for query batching, with three cross-query optimizations: trie caching, sub-expression sharing via spools, and shared-prefix execution. On synthetic graph workloads the system achieves up to 4x speedup with per-query cost dropping 9.5x as batch size grows. On TPC-H cyclic self-joins, WCOJ achieves 3.1x speedup and enables queries that binary joins cannot complete within memory limits. We also characterize failure modes: on FK cycles with highly selective closing predicates, binary joins outperform WCOJ by up to 3.6x.

1. Introduction

Standard query engines process joins in pairs [1]. For acyclic queries this works well, but for *cyclic* queries, no pairwise join ordering avoids intermediate blowup [2, 3].

Consider the triangle query, a fundamental motif in graph analytics:

$$Q_{\triangle}(a, b, c) \leftarrow R(a, b), S(b, c), T(c, a)$$

A binary plan that first computes $R \bowtie S$ on b produces all 2-hop paths (a, b, c), which can be $O(|E|^2)$ for graphs with high-degree hub nodes, before filtering with T. The final output (actual triangles) may be orders of magnitude smaller. Worst-case optimal join (WCOJ) algorithms [3, 4, 5] resolve this by processing one variable at a time, intersecting candidate values across *all* participating relations simultaneously. The resulting runtime is bounded by the *AGM bound* [2], the information-theoretic maximum output size given the input cardinalities, rather than by intermediate result sizes.

Independently, *multi-query optimization* (MQO) [6, 7, 8] addresses redundant computation when multiple queries share common sub-expressions. MQO has been studied extensively for binary-join workloads, but WCOJ's variable-at-a-time execution creates different sharing opportunities: trie indices can be shared across queries, identical search trees can be traversed once, and shared variable prefixes can be computed once and distributed to per-query suffix executors. These opportunities remain unexplored.

In this paper, we present an integrated system within Apache Calcite [9] that combines WCOJ execution with multi-query optimization. Our contributions are:

1. **WCOJ in Calcite.** A hash-based WCOJ operator integrated into Calcite's Volcano optimizer via automatic cyclic join detection. No pre-sorted indices or schema changes required.
2. **The `Combine` operator and `MULTI()` syntax.** A new relational operator that holds N independent sub-queries as a single plan, with a SQL extension for declaring query batches.
3. **Cross-query optimizations.** Three optimizations that exploit WCOJ's structure: (a) trie caching across the batch, (b) sub-expression sharing via spools, and (c) shared-prefix execution that computes common variable bindings once. We evaluate on synthetic graph workloads and TPC-H cyclic joins, characterizing both speedups and failure modes.

We show that WCOJ is not always beneficial: on FK cycles with selective closing predicates, binary joins are 1.5x–3.6x faster. The value of the `Combine` framework is that it is agnostic to join strategy, letting the optimizer route each sub-query to the best approach.

---

2. Background and Related Work

2.1 Worst-Case Optimal Join Algorithms

The AGM bound [2] gives a tight upper bound on join output size given input cardinalities. For the triangle query, it yields $|Q_\triangle| \leq |E|^{3/2}$, strictly better than the $O(|E|^2)$ intermediate possible with binary joins. Ngo et al.'s Generic-Join [3] achieves this bound by processing one variable at a time, intersecting candidate values across all relations at each level:

```
GENERIC-JOIN(Relations R_1...R_n, Variables x_1...x_m):
    if m = 0: emit matching tuples; return
    candidates <- intersect pi_{x_1}(R_i) for all R_i mentioning x_1
    for each v in candidates:
        bind x_1 <- v
        GENERIC-JOIN(R_1...R_n, x_2...x_m)
```

2.2 Practical WCOJ Implementations

Veldhuizen's Leapfrog Triejoin [4] proved WCOJ is practical but requires pre-sorted tries. EmptyHeaded [14] achieves large speedups on graph queries using SIMD-accelerated trie intersection but also requires pre-built indices. Freitag et al. [5] showed that hash-based WCOJ can be built inside a general-purpose RDBMS (Umbra), constructing tries at query time with no pre-sorting. Their approach is the closest precursor to ours; we adopt the same hash-trie design but add multi-query optimization. The Ring [29] supports all variable orderings from a single compact index, reducing *intra-query* index redundancy; our trie caching reduces *inter-query* redundancy across a batch. Free

Join [15] unifies binary and WCOJ under a single framework. None of these systems consider cross-query sharing.

Table 1. Comparison of WCOJ implementations.

System	Index Type	Pre-built?	General SQL?	Hybrid w/ Binary?	Multi-Query?
Leapfrog Triejoin [4]	Sorted trie	Yes	No (Datalog)	No	No
EmptyHeaded [14]	Columnar trie	Yes	No (graph)	No	No
Umbra [5]	Hash trie	No	Yes	Yes	No
Free Join [15]	Free join trie	No	Yes	Yes (unified)	No
Ring [29]	Wavelet tree	Yes	No (triples)	No	No
This work	**Hash trie**	**No**	**Yes**	**Yes**	**Yes**

2.3 Multi-Query Optimization

MQO is typically formulated as a materialization selection problem [6, 7, 10, 23, 25], which is NP-hard. We take a different approach, exploiting WCOJ's structure directly rather than solving a selection problem.

Operator-based approaches. Roy et al. [8] showed MQO can be implemented as a Volcano extension for binary-join plans. Tu et al. [11] proposed ψ-operators that algebraically combine multiple queries (up to 36x speedup); our `Combine` operator is similar in spirit but composes with WCOJ rather than binary joins. QPipe [22] shares work dynamically at runtime; our compile-time approach is less adaptive but cheaper to execute. In industry, Bruno et al. [26] and Roy et al. [27] implement computation reuse at scale, both exclusively on binary-join plans.

2.4 Apache Calcite

Apache Calcite [9] implements a hybrid Volcano/Cascades optimizer [19, 20] and serves as the query processing backbone for Hive, Flink, Druid, Trino, and many other systems. It provides extensible relational operators, a cost-based planner, code generation via Linq4j, and a pluggable SQL parser. Our extensions leverage all four components while preserving backward compatibility.

2.5 The Unexplored Intersection

No prior work has combined WCOJ with MQO. WCOJ's variable-at-a-time execution creates sharing opportunities unavailable to binary-join planners: tries can be shared across queries, identical search-space traversals can be deduplicated, and shared variable prefixes can be factored out. Section 5 describes the three optimizations that exploit these opportunities.

---

3. WCOJ Integration in Calcite

Figure 1 shows the system architecture. The new components (blue) are: (1) `MULTI()` SQL syntax and `Combine` node for query batching, (2) `EnumerableWCOJRule` with GYO cyclicity detection, (3) `EnumerableWCOJ` with hash tries, (4) `EnumerableCombine` with a shared `TrieCache`, and (5) sharing rules for sub-expression spooling and prefix execution.

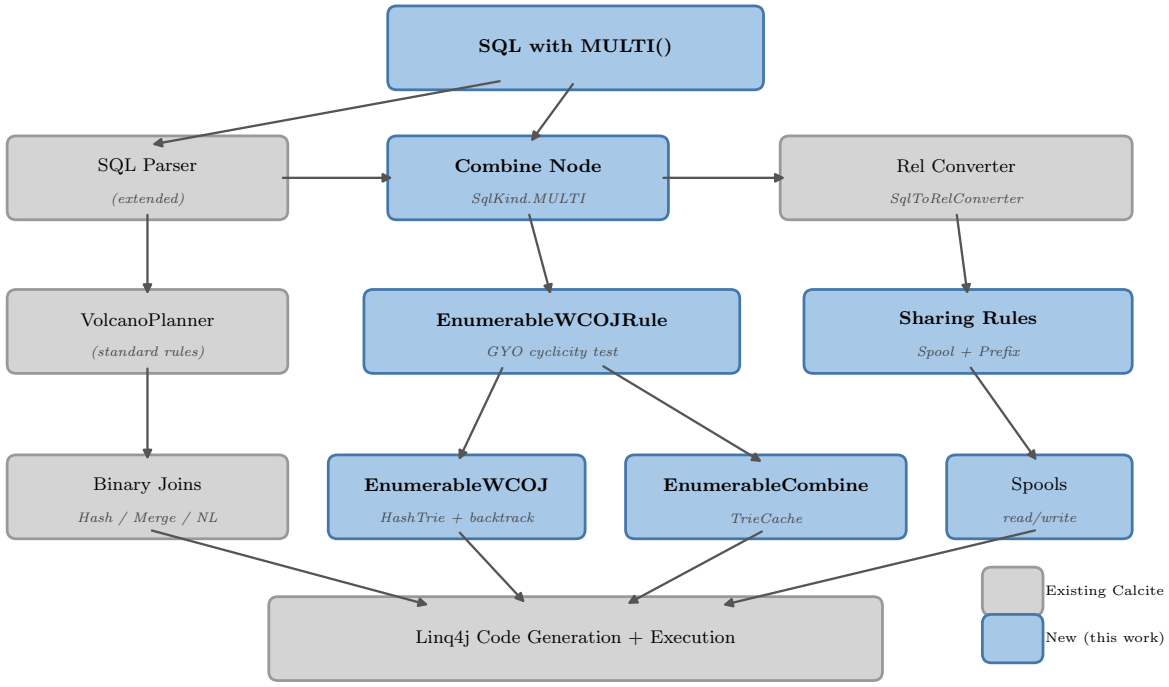

Figure 1: System architecture. Blue components are new; grey components are existing Calcite infrastructure.

3.1 Operator Design

Our WCOJ operator takes $N \geq 3$ inputs and processes them simultaneously via multi-way intersection, unlike binary join operators that combine exactly two inputs at a time. We implement this as `EnumerableWCOJ`, a physical operator in Calcite's enumerable convention.

Join Variables. The operator is parameterized by a list of `JoinVariable` objects, each representing an equivalence class of columns across inputs. For the triangle query with inputs $R(a, b)$, $S(b, c)$, $T(c, a)$:

- $v_0 = \{(R, a), (T, a)\}$: the shared variable a
- $v_1 = \{(R, b), (S, b)\}$: the shared variable b
- $v_2 = \{(S, c), (T, c)\}$: the shared variable c

3.2 Cyclic Query Detection

Detecting cyclic join patterns requires flattening the binary join tree so all predicates are visible simultaneously, then testing the resulting join graph for cycles.

Calcite represents joins as a binary tree of `LogicalJoin` nodes. `JoinToMultiJoinRule` (a standard Calcite rule) collapses this into a single `MultiJoin` with N inputs and one combined equi-join condition, making the global join structure accessible.

From this combined condition, we extract equi-join predicates and group co-equated fields into equivalence classes using Union-Find. Each equivalence class spanning two or more inputs becomes a `JoinVariable` (a hyperedge in the join hypergraph). Cyclicity is tested using *GYO reduction* [21]. The algorithm repeatedly removes *ear* hyperedges: a hyperedge H is an ear if every vertex in H shared with other hyperedges is contained within a single other hyperedge (the *witness*). If all hyperedges are removed, the query is acyclic; otherwise `EnumerableWCOJRule` fires.

For the triangle query, the hyperedges are $\{a, b\}$, $\{b, c\}$, $\{c, a\}$. Consider $\{a, b\}$: vertex a appears in $\{c, a\}$ and vertex b appears in $\{b, c\}$, so the shared vertices $\{a, b\}$ are not contained in any single other hyperedge. No hyperedge is an ear, so GYO reduction removes nothing and the triangle is correctly identified as cyclic.

3.3 Multi-Level Hash Tries

Our WCOJ implementation uses a purpose-built `HashTrie`: a recursive hash map where each level corresponds to a join variable in the global variable ordering:

```
HashTrie for R(a, b):
  root
  +-- a=1 --> { b=2: [row(1,2)], b=5: [row(1,5)] }
  +-- a=2 --> { b=3: [row(2,3)] }
  +-- a=3 --> { b=1: [row(3,1)] }
```

The trie supports three operations:

- **`build(source, extractors)`**: Scans the input once, inserting each row by extracting keys at each level. Time: $O(|R| \cdot d)$ where d is the number of levels.
- **`getKeysAtLevel(level, prefix)`**: Navigates using the prefix of prior bindings and returns distinct keys at the target level. Time: $O(d)$ navigation + $O(|\text{keys}|)$ enumeration.
- **`probe(keyValues)`**: Full probe with all levels specified, returns matching rows. Time: $O(d)$.

We use hash-based intersection rather than sorted-merge, following Freitag et al. [5]. Hash tries do not support sorted enumeration but can be built at query time without pre-sorted input.

3.4 WCOJ Enumerator

The `WCOJEnumerator` implements backtracking search over the global variable ordering. Each call to `moveNext()` finds the next complete binding and yields matching rows:

```
moveNext():
    level <- 0
    // Descend: bind one variable per level
    while level >= 0 and level < m:
        candidates <- intersect projections of all relations at level
```

```
        if advance to next candidate at level:
            level++
        else:
            level--    // backtrack
    if level < 0: return false   // exhausted

    // Yield result, then backtrack for next call
    emit cross-product of matching rows
    level--
    backtrack until next valid complete binding or exhausted
```

Backtracking is essential: a candidate at level k may have no valid continuations at $k+1$ even though other candidates at level k do.

The candidate computation at each level implements the intersection from Generic-Join:

$$\text{candidates}(x_k) = \bigcap_{R_i \ni x_k} \pi_{x_k} \left(\sigma_{x_1 = v_1, \ldots, x_{k-1} = v_{k-1}} (R_i) \right)$$

Each trie lookup navigates using the prefix of already-bound variables, returning only values consistent with all prior bindings.

Variable ordering. The current implementation uses the order in which equivalence classes are discovered during join graph analysis (Section 3.2), which follows the syntactic order of join predicates. This ordering is not optimized for single-query performance or cross-query prefix sharing; adaptive ordering is future work (Section 8, item 2).

3.5 Cost Model

The cost of `EnumerableWCOJ` has a lower bound of:

$$C_{\text{WCOJ}} \geq C_{\text{build}} + C_{\text{enum}} = \sum_{i=1}^{n} |R_i| + |Q(D)|$$

where C_{build} accounts for trie construction (linear scan of each input) and C_{enum} accounts for result enumeration. This lower bound captures the two unavoidable costs (reading each input once and producing each output tuple) but omits the per-level intersection cost of the WCOJ search:

$$C_{\text{search}} = \sum_{k=1}^{m} \sum_{(v_1, \ldots, v_{k-1}) \in \text{valid}} \min_{R_i \ni x_k} |\pi_{x_k} (\sigma_{x_1 = v_1, \ldots, x_{k-1} = v_{k-1}} (R_i))|$$

This is a lower bound. When C_{search} is small relative to $C_{\text{build}} + C_{\text{enum}}$ (self-join cycles with large output), it approximates the true cost well. When C_{search} dominates (FK cycles where a variable has high fan-out but low final selectivity), WCOJ can be slower than binary joins. Section 7.2.4 provides empirical evidence for both cases.

---

4. The Combine Operator and Multi-Query Framework

4.1 Motivation

Even with WCOJ, a batch of N structurally similar queries builds N identical trie structures and traverses the same search space N times. The `Combine` operator makes these redundancies visible to the optimizer by presenting all N queries as a single multi-root plan. `Combine` is agnostic to join strategy: sub-queries can use WCOJ, binary joins, or any other physical operator.

4.2 SQL Extension: MULTI()

We extend Calcite's SQL parser with a `MULTI` construct that declares a batch of queries for joint optimization:

```
MULTI(
  (SELECT e1.src, e1.dst, e2.dst
   FROM edges e1, edges e2, edges e3
   WHERE e1.dst = e2.src AND e2.dst = e3.src AND e3.dst = e1.src),
  (SELECT e1.src, e2.src, e3.src
   FROM edges e1, edges e2, edges e3
   WHERE e1.dst = e2.src AND e2.dst = e3.src AND e3.dst = e1.src),
  ...
)
```

The parser recognizes `MULTI` as a new keyword and produces a `SqlCall` with `SqlKind.MULTI`. During SQL-to-RelNode conversion, each sub-query is independently converted to a relational expression, and all are wrapped in a single `Combine` node.

4.3 The Combine Relational Operator

`Combine` is a new `AbstractRelNode` in Calcite's relational algebra that holds N independent sub-queries as children. Its key design properties:

Row type. A single `Combine` output row is a struct where each field is the result list from one sub-query, fitting Calcite's existing type system.

Cost model. `Combine` itself has minimal self-cost ($\sum_i |R_i| \times 0.01$ CPU). The optimizer evaluates the cumulative cost through children, allowing optimization rules to improve individual queries or exploit cross-query sharing.

Relationship to existing operators. Standard Calcite has no multi-root operator. The closest analog is `UNION ALL`, but `Combine` preserves independent result sets without requiring compatible schemas. This is essential for MQO: the optimizer can see all queries simultaneously and identify sharing opportunities that are invisible when queries are optimized in isolation.

4.4 Physical Implementation

`EnumerableCombine` implements code generation for the `Combine` operator. During `implement()`, it:

1. Creates a shared `TrieCache` instance and stores it on the `EnumerableRelImplementor`
2. Visits each child, converting each `Enumerable` result to a `List`
3. Packs all lists into a single struct row

4. Returns a singleton `Enumerable` containing that struct

The `TrieCache` creation in step 1 is the critical bridge for cross-query optimization: it provides a shared context that child WCOJ operators use to avoid redundant trie construction.

---

5. Cross-Query Optimizations

When multiple WCOJ queries execute within a `Combine`, three complementary optimizations eliminate redundant work. These optimizations target different levels of redundancy: trie caching shares *data structures* (the hash tries built from input relations), sub-expression sharing shares *sub-plans* (entire branches of the relational operator tree, such as a WCOJ node and its input scans), and prefix sharing shares *computation* (the backtracking search over shared join variables).

Optimization	What is shared	Enabled by
Trie caching	Hash trie data structures	Always active in `Combine`
Sub-expression sharing	Relational sub-plans (operator tree branches)	Planner rule (opt-in)
Prefix sharing	WCOJ search-space traversal	Planner rule (opt-in)

Trie caching is always active when the `Combine` operator is used. Sub-expression sharing and prefix sharing are additive planner rules that can be enabled independently.

5.1 Trie Caching

Problem. Each WCOJ operator independently builds a hash trie from its inputs. When two WCOJ operators join the same relation on the same key, they build identical tries.

Solution. A per-execution trie cache maps each `(input, keyIndex)` pair to a shared trie, built on first access. The cache uses object identity, so its effectiveness depends on whether two WCOJ operators reference the *same* input object or merely *equivalent* inputs:

1. **Intra-operator sharing (always active, no spooling needed).** Within a single WCOJ operator, a self-join references the same table scan object multiple times (e.g., a triangle query over `edges` uses one scan object as all three inputs). Since these references share the same object, the identity-based cache hits and the trie is built once. This is the sharing available in plain Combine mode.
2. **Cross-operator sharing (requires spooling).** When two *sibling* WCOJ operators within a `Combine` each scan the same table, they produce *separate* scan objects — one per operator. Even though the scans read identical data, the identity-based cache sees distinct objects and builds separate tries. To enable cross-operator cache hits, the sub-expression sharing rule (Section 5.2) wraps the shared scan in a spool, so both operators read from the *same* materialized object. This is the additional sharing that Combine-Share mode provides, and it explains the experimental gap between the two modes.

5.2 Sub-Expression Sharing via Spools

Problem. When the batch contains structurally identical sub-queries (e.g., the same triangle join appearing multiple times with different projections), each independently builds identical trie structures and traverses the same search space.

Solution. A planner rule identifies sub-plans that appear in two or more `Combine` children. The first occurrence is wrapped in a spool; subsequent occurrences are replaced with spool reads:

```
Before:                          After:
Combine                          Combine
+-- Project(a,b,c) <- WCOJ(...)  +-- Project(a,b,c) <- SPOOL(WCOJ(...))
+-- Project(x,y,z) <- WCOJ(...)  +-- Project(x,y,z) <- WCOJ(...)
+-- Project(a,b,c) <- WCOJ(...)  +-- Project(a,b,c) <- READ_SPOOL
```

The rule skips leaf table scans (already handled by trie caching) and the `Combine` root.

Memory considerations. Spooling materializes intermediate results in memory. For the TPC-H self-join triangle at SF=0.01, each spool holds up to 2.9M rows. The current implementation does not spill to disk.

5.3 Shared-Prefix Execution

Problem. Two WCOJ operators may enumerate the same variable prefix identically. For example, two triangle queries over the same graph might share all three join variables and differ only in their output projections. Without sharing, both operators independently traverse the same backtracking search space.

Solution. Detect shared prefixes at compile time via fingerprinting, compute the prefix once at runtime, and distribute the bindings to per-query suffix executors.

5.3.1 Join Variable Fingerprinting To compare variables across different WCOJ operators, we need a canonical representation that is independent of operator-local input numbering. Each variable is fingerprinted using the structural digest of each participating input and its field index:

$$\text{fingerprint}(v) = \text{sort}\left(\{(\text{digest}(R_{i_k}), f_k) \mid (i_k, f_k) \in v.\text{occurrences}\}\right)$$

The digest covers the full sub-plan (including filters and projections), so fingerprints match only when the underlying data and key structure are identical. If fingerprints match at positions $0, \ldots, K-1$, the WCOJ search over those variables produces identical bindings.

5.3.2 Prefix Group Detection Given N WCOJ operators, we build a *trie of fingerprint sequences* to find shared prefixes:

```
Input: WCOJ_0 with variables [fp_A, fp_B, fp_C]
       WCOJ_1 with variables [fp_A, fp_B, fp_D]
       WCOJ_2 with variables [fp_X, fp_Y]

Prefix trie:
    root
    +-- fp_A --> fp_B --> +-- fp_C  (WCOJ_0)
```

```
|                      +-- fp_D  (WCOJ_1)
+-- fp_X --> fp_Y             (WCOJ_2)
```

```
Result: PrefixGroup(depth=2, members=[0, 1])
```

A traversal of this trie finds *divergence points* (nodes with multiple children) and groups all descendant queries. Groups with depth ≥ 1 and $|\text{members}| \geq 2$ represent opportunities for shared-prefix execution.

The shared prefix depth depends on the variable ordering (Section 3.4).

5.3.3 Two-Phase Execution Once prefix groups are detected, grouped WCOJ operators are replaced with a two-phase execution strategy:

Phase 1: Prefix computation. The standard WCOJ algorithm is run truncated at depth K (the shared prefix depth), yielding variable *bindings* rather than result rows:

$$\text{WCOJ-PREFIX}(R_1 \dots R_n, x_1 \dots x_K) : \text{for each valid } (v_1, \dots, v_K), \text{yield } (v_1, \dots, v_K)$$

Phase 2: Suffix execution. For each prefix binding, a per-query suffix executor sets $x_1 = v_1, \dots, x_K = v_K$, initializes suffix variables $x_{K+1}, \dots, x_m$, and backtracks only within the suffix. A floor parameter prevents backtracking below the prefix boundary:

```
MOVE-NEXT-SUFFIX(floor):
    while currentLevel >= floor:
        if advanceAtLevel(currentLevel):
            propagate forward to deeper levels
        else:
            currentLevel <- currentLevel - 1
    if currentLevel < floor: return false  // prefix exhausted
```

The `floor` parameter prevents backtracking below the prefix boundary, confining suffix execution to the per-query divergent portion of the search space.

---

6. Formal Analysis

6.1 Correctness of Prefix Sharing

Theorem 1. *If two WCOJ operators share the same variable ordering for positions* $1, \dots, K$*, have matching fingerprints at those positions, and read from the same trie objects (via spooling), then they produce identical prefix bindings* $\{(v_1, \dots, v_K)\}$.

Proof sketch. At each level $k \leq K$, the candidates are:

$$\text{candidates}(x_k \mid v_1, \dots, v_{k-1}) = \bigcap_{R_i \ni x_k} \text{trie}_i.\text{getKeys}(k, \text{prefix}_i(v_1, \dots, v_{k-1}))$$

Matching fingerprints guarantee the same inputs participate at each level. Shared trie objects (via `TrieCache`) guarantee the same data is read. By induction: identical candidates at level k produce the same branches, so the search trees are isomorphic up to depth K. $\square$

6.2 Cost Analysis

Let P denote the cost of computing the shared prefix (iterating all valid $(v_1, \ldots, v_K)$ bindings), let S_i denote the suffix cost for query Q_i, and let N denote the number of queries in the prefix group.

Without sharing:

$$C_{\text{independent}} = N \cdot (P + \bar{S}) + N \cdot C_{\text{trie}}$$

where C_{trie} is the per-query trie construction cost.

With cross-query optimizations:

$$C_{\text{shared}} = P + N \cdot \bar{S} + C_{\text{trie}} + C_{\text{coord}}$$

where C_{coord} captures coordination overhead. The dominant component is forced result materialization: `EnumerableCombine` calls `.toList()` on each child, converting streaming results to in-memory lists. Secondary components include `TrieCache` lookups, spool materialization, and prefix binding dispatch.

Savings:

$$\Delta C = (N - 1) \cdot P + (N - 1) \cdot C_{\text{trie}} - C_{\text{coord}}$$

P dominates when the prefix covers most variables (queries differing only in projection or aggregation). C_{coord} grows as $O(|Q(D)|)$ due to result materialization (`.toList()` in `EnumerableCombine`), so the net benefit requires N large enough for the $(N-1) \cdot P$ savings to dominate.

---

7. Experimental Evaluation

7.1 Experimental Setup

We evaluate our system using a custom benchmark (`WCOJBenchmarkCli`) that compares four execution modes:

Mode	Description
baseline	Standard Calcite with binary hash joins, queries run sequentially
wcoj	WCOJ for each query, run sequentially (no multi-query optimization)
combine	`MULTI()` with WCOJ and trie caching (batched execution)
combine-share	`MULTI()` with WCOJ + trie caching + sub-expression sharing + prefix sharing
combine-binary	`MULTI()` with binary hash joins (batched, no WCOJ); used in TPC-H only

These modes form a 2x2 decomposition: {binary, WCOJ} × {sequential, batched}. The combine-binary mode isolates the contribution of batching from WCOJ.

Graph generation. We construct synthetic directed graphs where a small fraction of vertices (default 5%) are highly connected "hubs" with many incoming and outgoing edges, while the remaining vertices have few connections. This hub-and-spoke structure mimics the skewed degree distributions found in real-world graphs (social networks, web graphs). Hubs create large intermediate results during binary joins because any two-hop path through a hub fans out widely. We also add direct edges between non-hub vertices to guarantee that triangles exist in the graph, not just paths through hubs.

Query workload. Triangle queries $Q_{\triangle}(a, b, c) \leftarrow R(a, b), S(b, c), T(c, a)$ with 5 projection variations, wrapped in `MULTI()` for batched modes.

Hardware and software. All experiments are run on an Apple M4 Pro (14 cores) with 48 GB RAM, OpenJDK 21.0.9, and JVM heap limited to 2 GB (`-Xmx2g`). Calcite version is 1.41.0-SNAPSHOT.

7.2 Results

Timings are mean ± sample standard deviation over 10 iterations after warmup (3 warmup rounds for synthetic, 5 for TPC-H).

7.2.1 Scalability with Graph Size We fix the query batch size at $N = 5$ and vary graph size from 50 to 400 nodes, scaling edge counts proportionally with 5% hub nodes.

Table 2. Execution time (ms, mean ± sample std dev) and speedup vs. graph size, $N = 5$ triangle queries. Speedup over baseline shown in parentheses. †CV $> 20\%$.

Graph	Triangles	Baseline	WCOJ	Combine	Combine-Share
50 / 300	333	86 ± 8	58 ± 3 (1.5x)	37 ± 3 (2.3x)	37 ± 2 (2.3x)
100 / 800	999	170 ± 12	156 ± 41† (1.1x)	71 ± 7 (2.4x)	84 ± 9 (2.0x)
200 / 2000	1,854	265 ± 44	163 ± 11 (1.6x)	116 ± 15 (2.3x)	137 ± 24 (1.9x)
400 / 5000	3,525	772 ± 88	241 ± 31 (3.2x)	202 ± 24 (3.8x)	**192 ± 17** (**4.0x**)

Analysis. WCOJ's advantage grows with graph density (1.5x to 3.2x, Figure 3), as intermediate result explosion in binary joins worsens with graph size while WCOJ tracks output size. Combine adds consistent batching benefit (3.8x at $|V|$=400), and Combine-Share achieves the peak of **4.0x** once sharing savings outweigh coordination overhead.

7.2.2 Multi-Query Speedup We fix the graph at $|V| = 200$, $|E| = 2000$ and vary the number of batched triangle queries from $N = 5$ to $N = 20$.

Table 3. Execution time (ms) and per-query cost vs. batch size. Speedup over baseline in parentheses. †CV $> 20\%$.

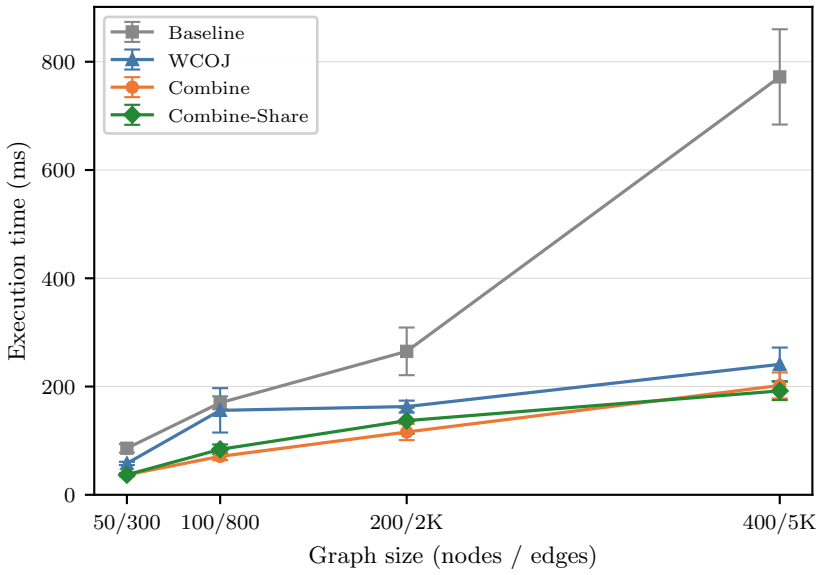

Figure 2: Execution time vs. graph size. Baseline grows super-linearly while WCOJ, Combine, and Combine-Share remain flat, with the gap widening at larger graph sizes. Error bars show ±1 sample standard deviation.

N	Baseline	WCOJ	Comb.	C-Share	/query: C	/query: CS
5	274 ± 33	182 ± 44†	110 ± 9 (2.5x)	116 ± 12 (2.4x)	22	23
10	559 ± 176†	297 ± 38	185 ± 34 (**3.0x**)	**165 ± 23** (**3.4x**)	19	**17**
20	597 ± 109	426 ± 64	224 ± 16 (**2.7x**)	247 ± 55† (2.4x)	**11**	12

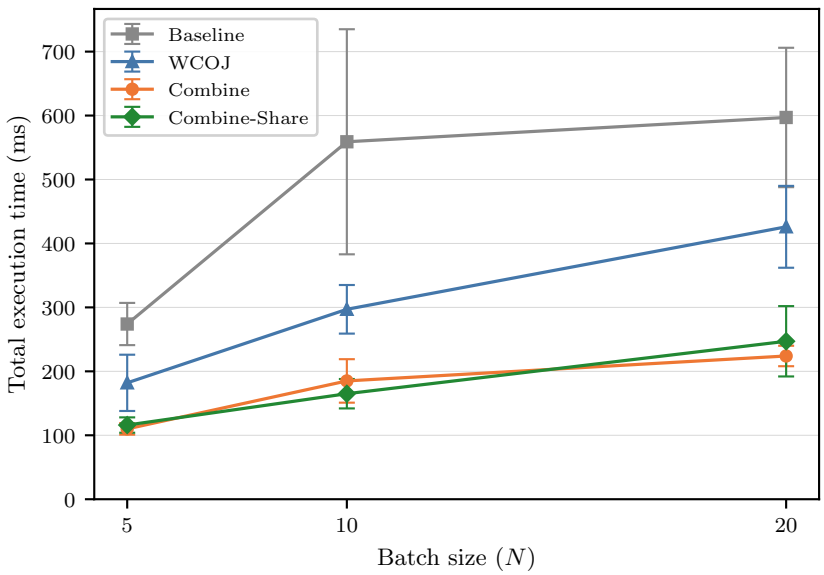

Figure 3: Total execution time vs. batch size. All 3 modes scale sub-linearly; Combine-Share leads at N=10 but plain Combine overtakes at N=20.

Analysis. Per-query amortized cost for `Combine` drops from 22 ms at $N = 5$ to **11 ms** at $N = 20$

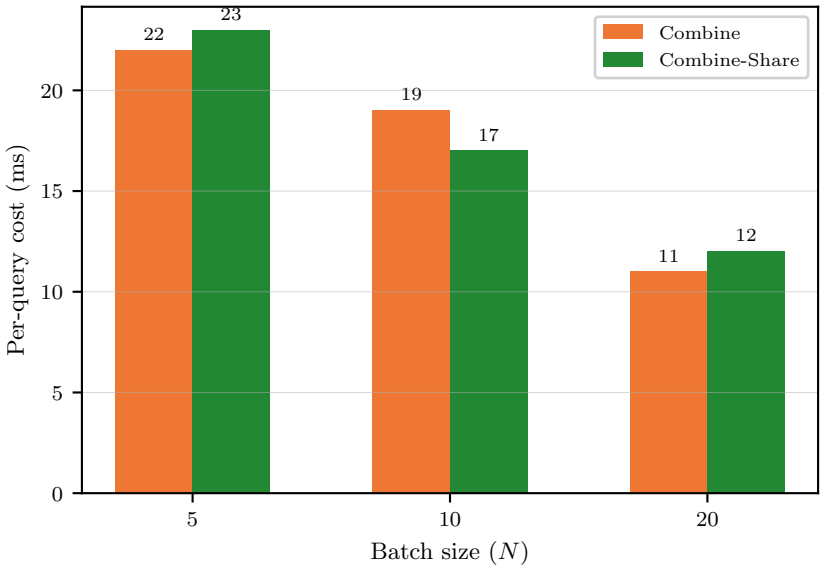

Figure 4: Per-query amortized cost vs. batch size. Both batched modes drop from about 22 ms to about 11 ms as batch size grows from 5 to 20.

Mode	V=400, N=5	Marginal gain	V=200, N=10	Marginal gain
Baseline	772 ms	-	559 ms	-
+ WCOJ	241 ms (3.2x)	3.2x	297 ms (1.9x)	1.9x
+ Combine	202 ms (3.8x)	+0.6x	185 ms (3.0x)	+1.1x
+ Combine-Share	**192 ms (4.0x)**	+0.2x	**165 ms (3.4x)**	+0.4x

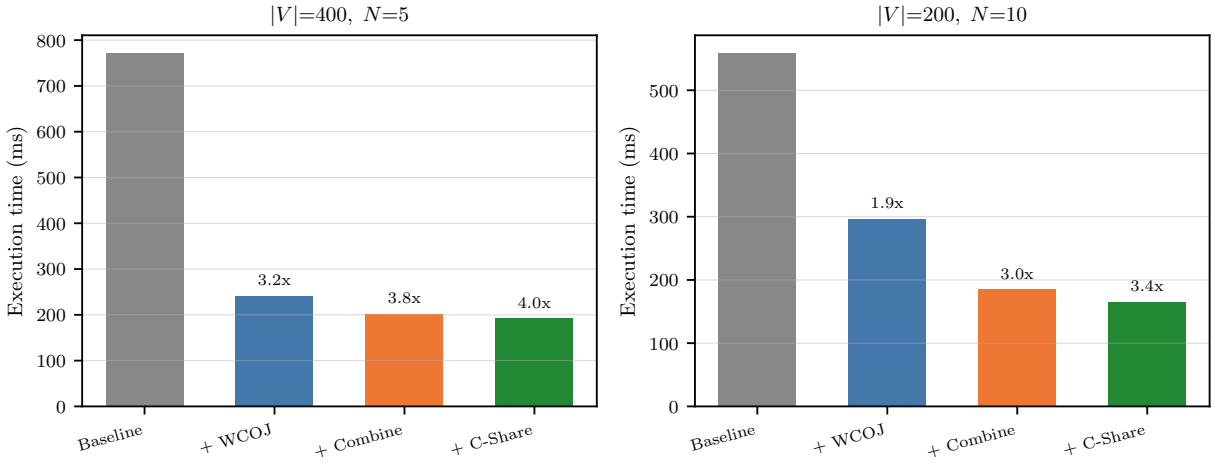

Figure 5: Optimization contributions waterfall at two operating points. WCOJ provides the dominant speedup; Combine and Combine-Share add incremental gains.

Analysis. WCOJ provides the dominant speedup (Figure 6). Combine adds batching benefit; Combine-Share adds further gains when there is genuine redundancy to eliminate (N=10 has each variation twice, yielding 1.12x over plain Combine). When coordination overhead exceeds sharing savings (e.g., N=20 in Table 3), plain Combine is preferable.

7.2.4 TPC-H Cyclic Joins The synthetic graph benchmarks above use self-joins on a single edge table, a pattern that maximizes WCOJ's advantage. To validate these results on realistic data, we construct cyclic join queries over the TPC-H schema (SF=0.01) and introduce a **combine-binary** mode (`MULTI()` batching with standard binary hash joins, no WCOJ) to isolate the contributions of WCOJ and batching.

Query design. Standard TPC-H queries are acyclic, but the schema naturally supports cyclic patterns. The `lineitem` table has multiple join keys (`l_orderkey`, `l_suppkey`, `l_partkey`), creating self-join triangles when joined on different key combinations:

```
-- Self-join triangle: orderkey-suppkey-partkey cycle
SELECT l1.l_orderkey, l2.l_suppkey, l3.l_partkey
FROM lineitem l1, lineitem l2, lineitem l3
WHERE l1.l_orderkey = l2.l_orderkey    -- same order
  AND l2.l_suppkey  = l3.l_suppkey     -- same supplier
  AND l3.l_partkey  = l1.l_partkey     -- same part (closes cycle)
```

We also test a self-join 4-cycle:

```
-- Self-join 4-cycle: suppkey-orderkey-partkey-orderkey
SELECT l1.l_orderkey, l2.l_suppkey, l3.l_partkey, l4.l_orderkey AS ok4
FROM lineitem l1, lineitem l2, lineitem l3, lineitem l4
WHERE l1.l_suppkey  = l2.l_suppkey     -- L1-L2: same supplier
  AND l2.l_orderkey = l3.l_orderkey    -- L2-L3: same order
  AND l3.l_partkey  = l4.l_partkey     -- L3-L4: same part
  AND l4.l_orderkey = l1.l_orderkey    -- L4-L1: same order (closes cycle)
```

`orderkey` appears in two join predicates (L2-L3 and L4-L1), linking different lineitem copies. We additionally test a foreign-key triangle (`lineitem` ⋈ `partsupp` ⋈ `supplier`) and two FK-based cyclic queries.

Table 5. TPC-H cyclic join performance (SF=0.01, $N = 5$ projection variations, 10 iterations, 5 warmup rounds). Mean ± sample std dev in ms. Speedup over baseline in parentheses. †High variance (CV > 20%). *Out of memory with 4 GB heap; the 4-way binary join intermediate exceeds available memory.

Query	Rows	Base	C-Bin	WCOJ	Comb.	C-Share
SJ △	2.9M	6102 ± 1349†	3546 ± 226 (1.7x)	2998 ± 425 (2.0x)	3884 ± 506 (1.6x)	**2511 ± 82 (2.4x)**
SJ □	6.0M	128311 ± 30538†	OOM*	**40772 ± 534 (3.1x)**	43889 ± 421 (2.9x)	46343 ± 1499 (2.8x)
FK △	301K	928 ± 109	599 ± 49 (1.5x)	881 ± 255† (~1.0x)	741 ± 148 (1.3x)	**573 ± 63 (1.6x)**

Table 6. TPC-H FK queries where WCOJ *underperforms* baseline (SF=0.01, $N = 5$, 10 iterations). Mean ± sample std dev in ms.

Query	Rows	Base (ms)	WCOJ (ms)	Ratio
FK □ (c-o-l-s)	11,665	860 ± 30	1,314 ± 41	**1.5x slower**
FK ♢ (c-o-l-s-n)	11,665	2,237 ± 25	7,950 ± 107	**3.6x slower**

Combine and Combine-Share modes show the same regression on these queries (1,350 ms and 1,359 ms for FK rectangle; 8,287 ms and 8,471 ms for FK diamond); batching does not recover the WCOJ penalty when the join algorithm itself is the bottleneck.

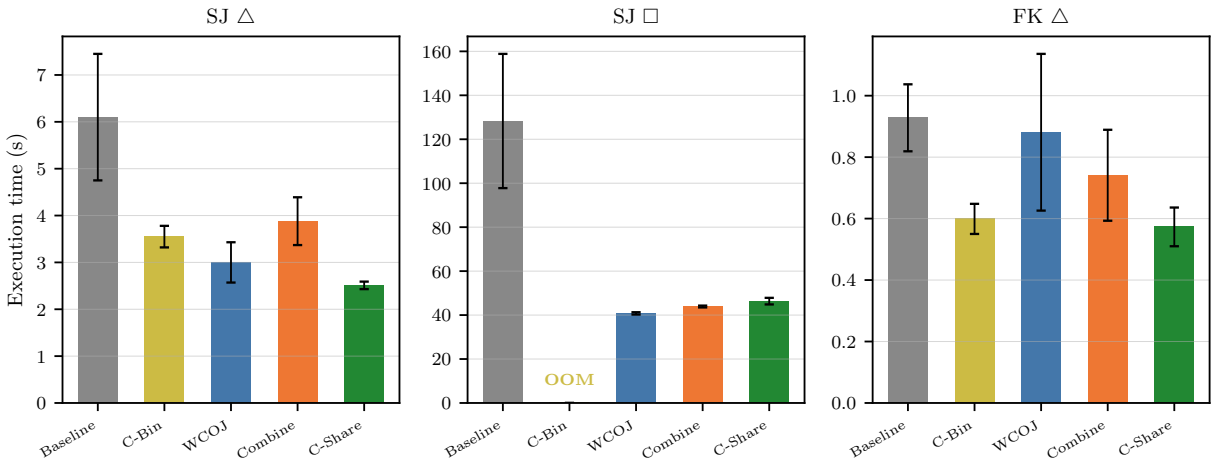

Figure 6: TPC-H cyclic join performance across three query shapes (separate y-axes). Self-join 4-cycle causes OOM for combine-binary; WCOJ and Combine-Share provide the largest gains on self-join patterns.

Analysis (Figure 7). *When WCOJ wins:* Self-join queries show clear benefits. The 4-cycle achieves **3.1x** speedup, and binary joins OOM on this query while WCOJ completes in 41s. The combine-binary column isolates the batching contribution (1.7x on the triangle), with WCOJ providing the rest.

When WCOJ loses: FK queries (Table 6) show WCOJ 1.5x–3.6x slower. The closing predicates on these cycles use low-cardinality keys (`nationkey`, 25 values), so binary joins prune candidates early and efficiently. WCOJ pays intersection cost at every level regardless. This is the C_{search} gap from Section 3.5.

Combine overhead: `EnumerableCombine` forces result materialization (`.toList()`), making plain Combine 30% slower than sequential WCOJ on the self-join triangle. Combine-Share recovers by sharing prefix computation, but not on the 4-cycle where coordination overhead on 6M rows dominates.

7.2.5 Limitations The experiments are subject to several constraints that bound the scope of the claims:

- **Scale factor.** TPC-H experiments use SF=0.01 (~60 MB). At larger scale factors, both the WCOJ speedups on self-join queries and the regressions on FK queries are expected to grow, but the crossover point between beneficial and harmful WCOJ application has not been characterized beyond SF=0.01.
- **In-memory spools.** Sub-expression sharing via spooling requires holding materialized intermediate results in memory. The current implementation does not spill to disk; queries that exceed heap will OOM rather than degrade gracefully.
- **Single machine.** All experiments run on a single M4 Pro. The framework's interaction with distributed query processing (e.g., Calcite over Flink or Trino) has not been evaluated.
- **Homogeneous batch structure.** The workload uses N projection variants over the same join structure. Workloads with structurally diverse queries across a batch would see less benefit from sub-expression sharing and prefix execution.

---

8. Conclusion

We presented extensions to Apache Calcite that combine WCOJ with multi-query optimization: the `Combine` operator and `MULTI()` syntax for query batching, and three cross-query optimizations (trie caching, sub-expression sharing, shared-prefix execution).

The system achieves up to **4.0x** speedup on synthetic cyclic queries, with per-query cost dropping 9.5x as batch size grows. On TPC-H self-joins, WCOJ achieves **3.1x** speedup and enables queries that binary joins cannot complete within memory limits. WCOJ is not universally beneficial: FK cycles with selective closing predicates cause 1.5x–3.6x regressions. The value of `Combine` is that it is agnostic to join strategy, providing a framework for routing each sub-query to the best approach.

For practitioners considering WCOJ, our results suggest it is most beneficial when the query is cyclic, intermediate results would be large relative to the final output, and the cycle-closing predicate is not highly selective. When these conditions are not met — as with FK joins where selective predicates let binary joins prune early — binary joins remain the better choice. Because all contributions are modular extensions that preserve backward compatibility, users can enable WCOJ selectively without disrupting existing workloads.

Several directions remain open. The largest bottleneck in prefix sharing is coordination overhead from materializing intermediate result sets; passing bindings via shared memory could reduce this cost significantly. Variable ordering currently follows syntactic predicate order, but an adaptive strategy that accounts for cross-query prefix sharing could improve both single-query and batched performance. Longer-term, integrating with Calcite's materialized view subsystem would enable persistent sharing across batches, and hybrid plans that route cyclic components to WCOJ while keeping acyclic components on binary joins would extend the framework's applicability. Finally, disk-spillable spools would allow the system to handle intermediate results that exceed heap memory.

References

[1] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, "Access path selection in a relational database management system," in *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, 1979, pp. 23–34. doi: 10.1145/582095.582099

[2] A. Atserias, M. Grohe, and D. Marx, "Size bounds and query plans for relational joins," *SIAM Journal on Computing*, vol. 42, no. 4, pp. 1737–1767, 2013. doi: 10.1137/110859440

[3] H. Q. Ngo, E. Porat, C. Re, and A. Rudra, "Worst-case optimal join algorithms," *Journal of the ACM*, vol. 65, no. 3, pp. 1–40, 2018. doi: 10.1145/3180143

[4] T. L. Veldhuizen, "Leapfrog Triejoin: A simple, worst-case optimal join algorithm," in *Proc. 17th International Conference on Database Theory (ICDT)*, Athens, Greece, 2014, pp. 96–106. Available: https://openproceedings.org/ICDT/2014/paper_13.pdf

[5] M. Freitag, M. Bandle, T. Schmidt, A. Kemper, and T. Neumann, "Adopting worst-case optimal joins in relational database systems," *Proceedings of the VLDB Endowment*, vol. 13, no. 11, pp. 1891–1904, 2020. doi: 10.14778/3407790.3407797

[6] T. K. Sellis, "Multiple-query optimization," *ACM Transactions on Database Systems*, vol. 13, no. 1, pp. 23–52, 1988. doi: 10.1145/42201.42203

[7] T. Kathuria and S. Sudarshan, "Efficient and provable multi-query optimization," in *Proc. 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*, 2017, pp. 53–67. doi: 10.1145/3034786.3034792

[8] P. Roy, S. Seshadri, S. Sudarshan, and S. Bhobe, "Efficient and extensible algorithms for multi query optimization," *ACM SIGMOD Record*, vol. 29, no. 2, pp. 249–260, 2000. doi: 10.1145/335191.335419

[9] E. Begoli, J. Camacho-Rodriguez, J. Hyde, M. J. Mior, and D. Lemire, "Apache Calcite: A foundational framework for optimized query processing over heterogeneous data sources," in *Proc. 2018 ACM International Conference on Management of Data (SIGMOD)*, Houston, TX, 2018, pp. 221–230. doi: 10.1145/3183713.3190662

[10] S. Zinchenko and D. Ponomaryov, "The selection problem in multi-query optimization: A comprehensive survey," *arXiv preprint arXiv:2412.11828*, 2025. doi: 10.48550/arXiv.2412.11828

[11] Y. Tu, M. Eslami, Z. Xu, and H. Charkhgard, "Multi-query optimization revisited: A full-query algebraic method," in *Proc. IEEE International Conference on Big Data*, 2022, pp. 252–261. doi: 10.1109/BigData55660.2022.10020338

[14] C. R. Aberger, A. Lamb, S. Tu, A. Notzli, K. Olukotun, and C. Re, "EmptyHeaded: A relational engine for graph processing," *ACM Transactions on Database Systems*, vol. 42, no. 4, pp. 20:1–20:44, 2017. doi: 10.1145/3129246

[15] Y. R. Wang, M. Willsey, and D. Suciu, "Free Join: Unifying worst-case optimal and traditional joins," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, Article 150, 2023. doi: 10.1145/3589295

[19] G. Graefe and W. J. McKenna, "The Volcano optimizer generator: Extensibility and efficient search," in *Proc. 9th IEEE International Conference on Data Engineering (ICDE)*, Vienna, Austria, 1993, pp. 209–218. doi: 10.1109/ICDE.1993.344061

[20] G. Graefe, "The Cascades framework for query optimization," *IEEE Data Engineering Bulletin*, vol. 18, no. 3, pp. 19–29, 1995.

[21] G. Gottlob, N. Leone, and F. Scarcello, "Hypertree decompositions and tractable queries," *Journal of Computer and System Sciences*, vol. 64, no. 3, pp. 579–627, 2002. doi: 10.1006/jcss.2001.1809

[22] S. Harizopoulos, V. Shkapenyuk, and A. Ailamaki, "QPipe: A simultaneously pipelined relational query engine," in *Proc. 2005 ACM SIGMOD International Conference on Management of Data*, 2005, pp. 383–394. doi: 10.1145/1066157.1066201

[23] S. Finkelstein, "Common expression analysis in database applications," in *Proc. 1982 ACM SIGMOD International Conference on Management of Data*, 1982, pp. 235–245. doi: 10.1145/582353.582400

[25] A. Jindal, K. Karanasos, S. Rao, and H. Patel, "Selecting subexpressions to materialize at datacenter scale," *Proceedings of the VLDB Endowment*, vol. 11, no. 7, pp. 800–812, 2018. doi: 10.14778/3192965.3192971

[26] N. Bruno, J. Debrodt, C. Song, and W. Zheng, "Computation reuse via fusion in Amazon Athena," in *Proc. 38th IEEE International Conference on Data Engineering (ICDE)*, 2022, pp. 1756–1767. doi: 10.1109/ICDE53745.2022.00166

[27] A. Roy, A. Jindal, P. Gomatam, X. Ouyang, A. Gosalia, N. Ravi, S. Mann, and P. Jain, "SparkCruise: Workload optimization in managed Spark clusters at Microsoft," *Proceedings of the VLDB Endowment*, vol. 14, no. 12, pp. 3122–3134, 2021. doi: 10.14778/3476311.3476388

[29] D. Arroyuelo, A. Gómez-Brandón, A. Hogan, G. Navarro, J. Reutter, J. Rojas-Ledesma, and A. Soto, "The Ring: Worst-case optimal joins in graph databases using (almost) no extra space," *ACM Transactions on Database Systems*, vol. 49, no. 2, Article 5, pp. 5:1–5:45, 2024. doi: 10.1145/3644824